

Facilitator Outline — Claude Code Sub-Agents (30 min, Applied AI Engineers)

Assets: slide deck · demo repo github.com/poornimasundararaman/sub-agent-pr-review · one-pager

Pre-session checklist (do before anyone joins)

- Clone the repo fresh; confirm **claude** starts and **/review** works end-to-end once. Note how long the run actually takes.
- Have two terminal windows ready: one with Claude Code in the repo, one showing **CLAUDE.md** and **.claude/commands/review.md**.
- Keep a completed review report from your dry run as backup in case the live run stalls.
- Optional pivot prep: have **gh pr diff <n> > pr.diff** ready if an attendee volunteers a real PR.

Run of show

Time	Segment	Slides	Demo
0:00–2:00	Open + why this session	1–2	—
2:00–5:00	The problem	3	—
5:00–9:00	The pattern	4–5	—
9:00–10:00	Demo setup — launch /review now	6	Kick off
10:00–15:00	Narrate while specialists run	—	Running (~5 min)
15:00–19:00	Walk the findings	—	Results
19:00–22:00	Templatize with CLAUDE.md	7	Show file
22:00–25:00	When / when not	8	—
25:00–29:00	Close question + discussion	9	—
29:00–30:00	Activation ask	—	—

Talk track by segment

0:00 — Open (slides 1–2)

"In 30 minutes you'll be able to explain the orchestrator/sub-agent pattern to a customer, build one live in Claude Code, and qualify where it fits — and where it doesn't." Set the contract: short concept block, then most of the time is a live demo you can rerun in any customer meeting.

2:00 — The problem (slide 3)

One reviewer, three domains, no consistency. "Ask a single long-context pass to hold logic, test coverage, and security simultaneously — none of it gets specialist attention, and no two runs weight

criteria the same way." Land the customer framing: this is the same failure mode as a human senior engineer reviewing alone at 6pm.

5:00 — The pattern (slides 4–5)

Decompose → Specialize → Synthesize. Definition to say verbatim: "A sub-agent is a specialized Claude instance launched by an orchestrator via the Task tool to execute one focused task independently, in parallel with peers, before returning a structured result." Hit the three wins: focus (role-specific reasoning), parallelization (~12 min sequential → ~5 min), task-scoped isolation (each returns a structured result; orchestrator synthesizes). Don't go deeper here — the demo proves it.

⚠ **Customer friction — raise it yourself before they do:** each specialist re-reads the full diff, so token spend is roughly 3× a single pass. Teach the response: "You're paying 3× tokens for 3× specialist coverage in a third of the time — price it against the cost of a missed live Stripe key in production, not against one prompt."

9:00 — DEMO LANDS HERE (slide 6, then leave slides)

Launch first, explain second — the run takes ~5 min, so start it before narrating.

1. Show the repo: "Three files matter — `CLAUDE.md`, the `/review` command, and `sample_pr.diff`, a Flask + Stripe payments service with bugs planted in every specialist's domain."
2. Type `/review`. As the three Task spawns appear: "That's the orchestrator delegating. Logic Reviewer, Test Writer, Security Auditor — running in parallel, right now."

While it runs (10:00–15:00), fill the wait deliberately:

- Open `CLAUDE.md` in the second window — walk the roles and orchestrator instructions ("spawn all three in parallel, synthesize, flag P0 security at top").
- Open `.claude/commands/review.md` — "the slash command is just a file; no subprocess management, no API plumbing."
- Mention pivotability: "`/review` accepts any diff — `git diff`, `gh pr diff`. I can run this on your PR in this meeting." This is the line AAI engineers reuse with prospects.

Walk the findings (15:00–19:00) — call out one headline per specialist to prove specialization is real:

- Logic: Stripe amount-unit mismatch; leaked DB connection; refund endpoint doesn't verify charge ownership.
- Security: hardcoded live Stripe key; SQL injection in two endpoints; shell injection; no auth anywhere.
- Tests: `/refund` completely untested; a test hitting a real DB; a test for a removed code path.

Punchline: "One reviewer holding all three domains misses some of these every time. Three specialists gave you three times the coverage — and it didn't take any longer than a single review."

⚠ **Customer friction in the demo itself:** (1) Parallel Task spawns burn through rate limits fast on lower-tier plans — a customer's first run may throttle where yours didn't. (2) Specialists share no context, so reports can contain overlapping or duplicate findings; point out that the orchestrator's synthesis step is where dedup happens. (3) Diffs over ~100KB approach context limits — the

answer is splitting by file, not abandoning the pattern. Naming these during the demo builds trust; getting surprised by them at the customer's desk doesn't.

If the live run stalls: switch to the backup report from your dry run, narrate it the same way, and note that's why you always dry-run before a customer demo.

19:00 — Templatize (slide 7)

"What you just watched isn't a script — it's a markdown file." Drop `CLAUDE.md` at repo root, Claude Code reads it every run: zero onboarding, consistent auditable output, adding a specialist is a copy-paste edit. The repo is the leave-behind; a customer can commit an adapted `CLAUDE.md` the same day.

⚠ Customer friction at adoption: the committed `CLAUDE.md` is where activation stalls. Common blockers: security review of what code Claude sees, "who owns this file" debates, and engineers treating AI findings as noise until the first real catch. Coach the room to anchor on one team, one repo, one committed file — not an org-wide rollout pitch.

22:00 — When / when not (slide 8)

Spend equal time on both columns — qualifying out builds credibility. Good fit: clear delegation points, parallelizable domains, failure isolation, specialized context per step. Not a fit: simple linear tasks, interdependent shared state, latency-sensitive paths, and the overhead tax — cost, rate limits, context limits. Say it plainly: "If a single prompt does it, use a single prompt." This slide is where the friction points raised earlier resolve — recap them as qualification criteria, not apologies.

25:00 — Close (slide 9)

Ask the success-signal question and let 2–3 people answer: **"Where in your current pipeline would you use a role-specialized sub-agent workflow — and where wouldn't you?"** A passing answer names a real account, assigns concrete specialist roles, explains why specialization beats one long-context call, and names at least one constraint unprompted. If answers are generic, push: "Which account? What are the three roles?"

29:00 — Activation ask

"The milestone is a confirmed run in a customer's repo — committed `CLAUDE.md` or a first review report. No run, no activation." Set the 30-day follow-ups: time-saved-per-task benchmark (capture baseline now) and count of teams with `CLAUDE.md` committed.

Friction quick-reference (objection → response)

Customer objection	Response	Raised in
"This costs 3× the tokens"	3× specialist coverage in a third of the time; compare against the cost of a missed P0, not a single prompt	Slide 5
"We hit rate limits immediately"	Known on lower tiers with parallel spawns; size the plan before the pilot, or run specialists in smaller batches	Demo

Customer objection	Response	Raised in
"The report repeats itself"	Specialists are isolated by design; dedup is the orchestrator's synthesis job — tune it in CLAUDE.md	Demo
"Our PRs are huge"	>100KB diffs approach context limits; split by file — a constraint, not a blocker	Demo
"Security won't approve this" / "who owns CLAUDE.md"	Start with one team, one repo, one committed file; the file itself is the audit trail	Slide 7
"Why not just one big prompt?"	For simple linear tasks — yes, exactly. That's the qualification slide	Slide 8

Scope guardrails (deflect, don't debate)

Out of scope — park these with "good question, separate session": multi-orchestrator / agent-to-agent handoffs, per-specialist prompt tuning, CI/CD wiring and MCP integrations, pricing and rate-limit deep-dives.

Timing flex

- Running long: compress slide 5 (the demo proves the same points) and cut discussion to one answer.
- Running short: take a volunteer's real diff and rerun **/review** live — strongest possible close.